ARQUITETURA DE COMPUTADORES · AULA 01

INTRODUÇÃO AO ASSEMBLY NASM

Da sintaxe MASM de 32 bits ao NASM de 64 bits em Linux.

TRANSIÇÃO

MASM E NASM EXPRESSAM OPERAÇÕES DE MÁQUINA COM SINTAXES DIFERENTES

CONCEITOS QUE PERMANECEM

Instruções, registradores, operandos, rótulos, comentários e organização de dados e código.

O QUE MUDA

Palavras-chave, diretivas e convenções do montador e do ambiente de execução.

IDEIA Aprender outro montador é reconhecer a mesma lógica em um dialeto sintático diferente.

LINHA DE CÓDIGO

UMA INSTRUÇÃO PODE TER RÓTULO, OPCODE, OPERANDOS E COMENTÁRIO

```
inicio: mov rax, 1 ; código de syscall write
```

Rótulo

`inicio` nomeia uma posição do programa.

Opcode e operandos

`mov rax, 1` indica a operação e seus dados.

Comentário

`; ...` explica o trecho para quem lê o código.

MAPEAMENTO

DIRETIVAS E REGISTRADORES MUDAM ENTRE OS AMBIENTES

CONCEITO	MASM · X86 32 BITS	NASM · X86-64
Seção de dados	`SEGMENT`	`section .data`
Seção de código	`SEGMENT`	`section .text`
Registrador acumulador	`EAX`	`RAX`
Instrução de cópia	`MOV EAX, 1`	`mov rax, 1`

RAX tem 64 bits; EAX nomeia a parte baixa de 32 bits desse registrador.

DIRETIVAS

O MONTADOR INTERPRETA DIRETIVAS QUE A CPU NÃO EXECUTA

Instruções

`mov` e `syscall` viram operações executadas pela CPU.

Diretivas

`section`, `db`, `resb` e `global` orientam a montagem e a organização do programa.

PERGUNTA Quais linhas deixam de existir como texto após a montagem?

SEÇÕES

DADOS E INSTRUÇÕES FICAM EM SEÇÕES LÓGICAS DIFERENTES

SECTION .DATA

Declara dados inicializados, como uma string terminada por quebra de linha.

```
msg db "Hello world!", 10
```

SECTION .TEXT

Contém instruções e o ponto de entrada declarado com `global _start`.

O rótulo `_start` identifica onde a execução começa.

SYSCALL

LINUX X86-64 RECEBE A CHAMADA E OS ARGUMENTOS EM REGISTRADORES

REGISTRADOR	VALOR NO EXEMPLO	PAPEL
RAX	1	Número da chamada `write`
RDI	1	Descritor de saída padrão
RSI	endereço de `msg`	Endereço dos bytes a escrever
RDX	13	Quantidade de bytes

O número 60 em RAX solicita `exit`; RDI contém o código de saída.

HELLO WORLD

ESTE PROGRAMA ESCREVE A MENSAGEM E ENCERRA

```
section .data
    msg db "Hello world!", 10

section .text
    global _start
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 13
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

O roteiro indica execução em ambiente Linux de 64 bits, como mycompiler.io.

FLUXO DO PROGRAMA

DADOS SÃO DECLARADOS, REFERENCIADOS E ENVIADOS À SAÍDA

01 **Declarar**

db reserva e inicializa os bytes da mensagem.

02 **Preparar**

Registradores recebem chamada, descritor, endereço e tamanho.

03 **Executar**

`syscall` pede ao Linux que escreva; outra chamada encerra o processo.

SÍNTESE

A LÓGICA ASSEMBLY PERMANECE; A SINTAXE DEPENDE DO MONTADOR

- NASM usa diretivas como `section .data` e `section .text`.
- RAX estende EAX para 64 bits.
- Diretivas organizam a montagem; instruções são executadas pela CPU.
- Linux x86-64 usa `syscall` para solicitar serviços do sistema.

REFERÊNCIA

LEITURA DA AULA

Tanenbaum, A. S. *Organização Estruturada de Computadores*. 6ª ed. Registros de aula e guia de transição MASM/NASM.